```
    {}
    void lock()
    {
        check_for_hierarchy_violation();
        internal_mutex.lock();                    <-- 4
        update_hierarchy_value();                 <-- 5
    }
    void unlock()
    {
        this_thread_hierarchy_value=previous_hierarchy_value;   <-- 6
        internal_mutex.unlock();
    }
    bool try_lock()
    {
        check_for_hierarchy_violation();
        if(!internal_mutex.try_lock())            <-- 7
            return false;
        update_hierarchy_value();
        return true;
    }
};
thread_local unsigned long
    hierarchical_mutex::this_thread_hierarchy_value(ULONG_MAX);   <-- 8
```

The key here is the use of the `thread_local` value representing the hierarchy value for the current thread: `this_thread_hierarchy_value` ❶. It's initialized to the maximum value ❽, so initially any mutex can be locked. Because it's declared `thread_local`, every thread has its own copy, so the state of the variable in one thread is entirely independent of the state of the variable when read from another thread. See appendix A, section A.8, for more information about `thread_local`.

So, the first time a thread locks an instance of `hierarchical_mutex` the value of `this_thread_hierarchy_value` is `ULONG_MAX`. By its very nature, this is greater than any other value, so the check in `check_for_hierarchy_violation()` ❷ passes. With that check out of the way, `lock()` delegates to the internal mutex for the actual locking ❹. Once this lock has succeeded, you can update the hierarchy value ❺.